



中国矿业大学
CHINA UNIVERSITY OF MINING AND TECHNOLOGY

课程实验报告

基于 SAM 3.1 的开放词汇视频目标分割实验

模型结构、核心代码、运行流程与多提示词实验分析

课程名称:	图像处理与计算机视觉
任课教师:	邵志文
学生姓名:	卜炜珏
学号:	08231338
班级:	计科 3 班
完成日期:	2026 年 6 月

摘要

本次作业选用 Meta 官方开源的 SAM 3.1, 在一段足球短视频上完成开放词汇目标分割与跨帧跟踪, 并补充若干单帧图像提示实验。实验采用官方 SAM3 代码仓库, 模型权重从 ModelScope 的 `facebook/sam3.1` 仓库下载, 运行环境为 NVIDIA GeForce RTX 4090、Python 3.12 和 PyTorch 2.6.0+cu124。报告重点分析项目代码结构、核心推理入口、Object Multiplex 多目标跟踪机制、可视化输出流程和本地实验脚本, 并使用 `person`、`ball`、`player in red` 三组视频文本提示词完成对比实验。实验结果显示, SAM 3.1 能根据自然语言提示发现不同粒度的目标实例, 并在连续视频帧中保持较稳定的目标编号和分割结果; 单帧实验进一步展示了场景级、物体级和部件级提示的输出差异。

关键词: SAM 3.1; 开放词汇分割; 视频目标跟踪; Object Multiplex; PyTorch

目录

第一章 实验项目与研究背景	1
1.1 项目选择	1
1.2 实验目标	1
1.3 实验环境	1
第二章 SAM 3.1 项目代码结构解读	2
2.1 官方仓库结构	2
2.2 模型结构	3
第三章 核心代码分析	4
3.1 模型构建入口	4
3.2 Session 式视频推理	4
3.3 输出数据结构	5
3.4 Object Multiplex 跟踪逻辑	5
3.5 可视化流程	6
3.6 本地实验脚本	6
3.7 单帧图像实验脚本	7
第四章 实验环境与运行流程	7
4.1 环境配置命令	7
4.2 权重下载	8
4.3 实验运行命令	8
第五章 多组实验结果与对比分析	9
5.1 视频实验结果汇总	9
5.2 粗粒度类别提示: person	11
5.3 小目标提示: ball	11
5.4 细粒度属性提示: player in red	11
5.5 单帧图像补充实验	11
5.6 结果对比小结	12
第六章 问题处理与总结	13
6.1 运行中遇到的问题	13
6.2 实验结论	13
6.3 后续改进	13

第一章 实验项目与研究背景

1.1 项目选择

项目选用 Meta 的 SAM 3: Segment Anything with Concepts, 它是 SAM 系列的第三代统一分割基础模型, 也是本次课程作业中的图像处理深度学习开源项目。官方仓库位于 GitHub[Meta AI Research(2026a)], 项目页说明了模型定位并提供了示例演示 [Meta AI Research(2026b)]。本次实验的脚本、报告和结果截图整理在个人公开仓库中 [Bu(2026)], 官方源码和模型权重不随作业仓库上传。SAM 3 支持文本、点、框、mask 等多种提示方式。相比 SAM 2, SAM 3 的关键变化是增加了开放词汇概念分割能力: 用户输入简短文本短语后, 模型可以自动检测并分割相关实例。

SAM 3.1 是 2026 年 3 月发布的更新版本, 重点引入 Object Multiplex, 用共享记忆方式联合处理多个视频目标, 降低多目标跟踪中的重复计算 [Meta AI Research(2026)]。实验中直接加载 `sam3.1_multiplex.pt checkpoint`, 用文本提示词在足球视频帧序列中完成分割与跟踪。

1.2 实验目标

实验工作分为三块。工程部分是在本地 GPU 上配置独立 Conda 环境、下载权重并跑通 SAM 3.1 推理; 代码部分梳理仓库结构、模型构建入口、session 推理接口、Object Multiplex 跟踪逻辑、可视化工具和本地脚本改造; 结果部分以视频目标分割为主, 配合单帧图像提示实验, 观察粗粒度类别、小目标、属性约束和部件级提示的输出差异。

1.3 实验环境

依赖安装在独立环境 `sam31` 中, 没有写入 `base` 环境。由于本机 NVIDIA 驱动显示 CUDA 12.4, 实验没有直接使用官方 README 中的 CUDA 12.8 PyTorch 命令, 而是安装与当前驱动兼容的 PyTorch 2.6.0+cu124 预编译包 [PyTorch Contributors(2026)]。环境配置见表 1。

表 1 实验运行环境

项目	配置
操作系统	Linux
Conda 环境	sam31
Python	3.12.13
GPU	NVIDIA GeForce RTX 4090
GPU 显存	49140 MiB
PyTorch	2.6.0+cu124
TorchVision	0.21.0+cu124
CUDA Runtime	12.4
模型权重	sam3.1_multiplex.pt, 来自 ModelScope[ModelScope(2026)]

第二章 SAM 3.1 项目代码结构解读

2.1 官方仓库结构

官方仓库采用标准 Python 包结构。根目录提供说明文档、发布说明、包配置和示例资源，核心实现位于 `sam3/` 包内。主要目录见表 2。

表 2 SAM3 官方仓库结构说明

路径	作用
README.md	项目介绍、环境安装、基础图像和视频推理示例。
RELEASE_SAM3p1.md	SAM 3.1 发布说明，介绍 Object Multiplex、效率提升和新 checkpoint。
pyproject.toml	Python 包配置和基础依赖。
examples/	官方 notebook 示例，包括图像预测、视频预测、批量推理和 SAM 3.1 multiplex 示例。
assets/	官方示例图片、GIF、模型结构图和效率图。
sam3/model_	模型构建入口，封装图像模型、视频模型和 SAM
builder.py	3.1 multiplex predictor。
sam3/model/	核心模型模块，包括编码器、检测器、跟踪器、multiplex 跟踪和 session predictor。
sam3/sam/	SAM 系列基础模块，包括 prompt encoder、mask decoder 和 Transformer。

路径	作用
sam3/ visualization_ utils.py	将分割 mask、边界框和目标编号绘制到原始图像上的工具。
scripts/eval/	benchmark 评估、数据下载和格式转换脚本。

2.2 模型结构

SAM 3 由 detector 和 tracker 组成，二者共享视觉编码器。detector 负责根据文本、几何提示和图像特征发现目标实例，tracker 负责在视频帧之间传播目标记忆并输出连续的 masklet。官方结构如图 1 所示。

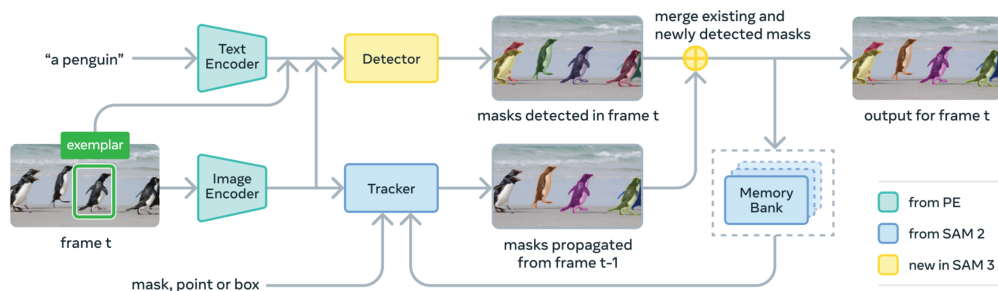


图 1 SAM 3 官方模型结构图。模型将视觉特征、文本提示和几何提示融合，用 detector 输出实例，再由 tracker 完成视频传播。

SAM 3.1 的 Object Multiplex 主要针对视频跟踪阶段。原始 SAM 3 对每个目标相对独立地处理记忆，目标数增加时容易产生重复计算；SAM 3.1 将多个对象组织到固定容量的 bucket 中联合传播，减少记忆读取与更新的重复开销。图 2 展示了官方的 multiplex 思路。

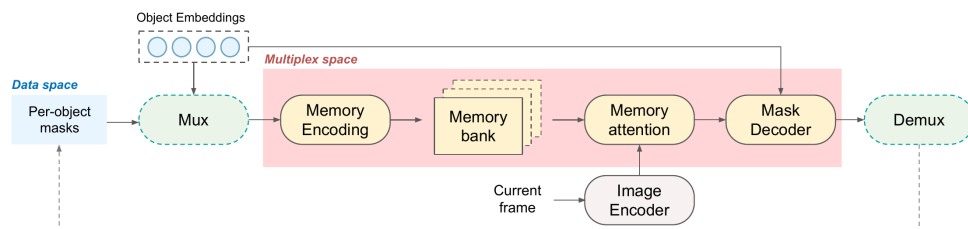


图 2 SAM 3.1 Object Multiplex 示意图。多个对象共享一组记忆处理流程，以提高多目标视频跟踪效率。

第三章 核心代码分析

3.1 模型构建入口

模型加载入口位于 `sam3/model_builder.py`。其中的 `predictor` 构造函数会构建 `tracker`、`detector` 和 `demo model`，最后封装为 SAM 3.1 multiplex 视频 predictor。

Listing 1 SAM 3.1 multiplex predictor 构建示例

```
1 from sam3.model_builder import build_sam3_multiplex_video_predictor
2
3 predictor = build_sam3_multiplex_video_predictor(
4     checkpoint_path="checkpoints/sam3.1/sam3.1_multiplex.pt",
5     use_fa3=False,
6     compile=False,
7     warm_up=False,
8     async_loading_frames=False,
9 )
```

这个入口依次完成模型构建、predictor 封装、权重加载和推理模式设置。它先调用 `build_sam3_multiplex_video_model` 创建底层跟踪模型，再构建 `detector` 封装器和交互式 tracking 封装器，形成可交互的视频分割模型。随后，函数从 checkpoint 读取权重，处理可能存在的键名前缀映射，最后将模型移动到 CUDA 并切换到 eval 模式。

实验里关闭了 `use_fa3`，因为 FlashAttention 3 是可选加速组件，对课程目标没有影响。默认 `max_num_objects=16` 和 `multiplex_count=16` 与官方 checkpoint 参数形状匹配，实验中尝试改为 8 会产生权重维度不一致。

3.2 Session 式视频推理

SAM3 视频 predictor 采用请求分发接口。外部调用不直接访问底层模型，而是向 predictor 发送带有 `type` 字段的请求。请求分发逻辑位于 `sam3/model/sam3_base_predictor.py`，常用请求包括 `start_session`、`add_prompt`、`propagate_in_video` 和 `close_session`。

Listing 2 文本提示和跨帧传播的核心调用

```
1 response = predictor.handle_request(
2     request={
3         "type": "add_prompt",
4         "session_id": session_id,
5         "frame_index": 0,
6         "text": "person",
7     }
```

```

8 )
9
10 outputs_per_frame = {0: response["outputs"]}
11 for response in predictor.handle_stream_request(
12     request={
13         "type": "propagate_in_video",
14         "session_id": session_id,
15     }
16 ):
17     outputs_per_frame[response["frame_index"]] = response["outputs"]

```

这种设计将交互式视频分割抽象为会话状态。模型只需在 session 初始化时加载帧序列，后续可以在不同帧追加文本、点或框提示，再将新的目标状态传播到全视频。

3.3 输出数据结构

每一帧的输出是一个字典，主要字段见表 3。

表 3 SAM 3.1 视频输出字段

字段	含义	后续用途
out_binary_masks	每个实例对应的二值分割 mask	叠加到原始帧形成可视化结果。
out_boxes_xywh	归一化边界框，格式为 x、y、w、h	绘制目标框，辅助观察目标位置。
out_probs	每个实例的置信度	在图上显示 score，帮助判断输出可靠性。
out_obj_ids	跟踪目标编号	观察同一对象在不同帧中的 id 是否稳定。

3.4 Object Multiplex 跟踪逻辑

Object Multiplex 的实现集中在 sam3/model/sam3_multiplex_tracking.py。从代码组织上看，它在普通视频跟踪状态之外维护额外的 tracker metadata、feature cache 和 generator state。多个对象会被组织到固定容量的 multiplex bucket 中，模型在一次传播中批量处理这些对象。

这个机制不改变上层调用方式，主要在内部减少多目标跟踪时的重复运算。使用者仍然只需要调用 add_prompt 和 propagate_in_video；在模型内部，多个目标的 memory 更新、mask 输出和后处理可以共享一部分计算路径。

3.5 可视化流程

可视化函数位于 `sam3 / visualization_utils.py`。本实验使用 `save_masklet_image` 将每帧输出保存为图片。该函数内部先读取原始帧，再调用渲染函数叠加 mask、边界框、object id 和置信度。

Listing 3 单帧可视化保存示例

```
1 save_masklet_image(
2     frame=str(frames[frame_idx]),
3     outputs=outputs_per_frame[frame_idx],
4     out_path=str(output_path),
5     alpha=0.5,
6     frame_idx=frame_idx,
7 )
```

3.6 本地实验脚本

本地脚本 `scripts/run_sam31_player.py` 把官方 notebook 中的推理逻辑抽出来，改成命令行可复现的实验。脚本流程如图 3 所示。

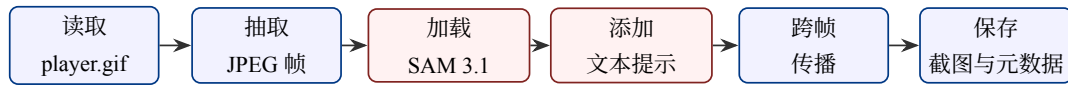


图 3 本地实验脚本推理流程。脚本将视频帧抽取、模型加载、文本提示、跨帧传播和结果保存组织为可重复命令。

脚本还增加了 `--output-prefix` 参数，避免不同提示词实验的图片、metadata 和日志相互覆盖。例如 `person` 对应 `sam31_player_person_summary.png`，`ball` 对应 `sam31_player_ball_summary.png`。

Listing 4 多提示词输出命名逻辑

```
1 parser.add_argument("--prompt", default="person", help="Text prompt.")
2 parser.add_argument(
3     "--output-prefix",
4     default=None,
5     help="Output filename prefix. Defaults to a prompt-based slug.",
6 )
7
8 def slugify_prompt(prompt: str) -> str:
9     slug = re.sub(r"[^a-z0-9]+", "_", prompt.lower()).strip("_")
10    return slug or "prompt"
11
12 output_prefix = args.output_prefix or f"sam31_player_{slugify_prompt(args.prompt)}"
13 summary_path = args.results_dir / f"{output_prefix}_summary.png"
```

```
14 metadata_path = args.results_dir / f"{output_prefix}_metadata.json"
```

这段改动的作用是把提示词转换为安全文件名前缀，并允许命令行手动指定输出名。实验运行三组 prompt 时，截图、metadata 和模型加载日志会分别写入 sam31_player_person、sam31_player_ball 和 sam31_player_red 前缀下，便于后续放入报告和复查。

3.7 单帧图像实验脚本

除了视频 predictor，本实验还补充了 scripts/run_sam31_image_cases.py。该脚本使用 sam3/model/sam3_image_processor.py 中的 Sam3Processor，在若干静态图片上执行文本提示分割。其核心调用更接近普通图像分割任务：先构建图像模型，再设置输入图像，最后输入文本 prompt 得到 masks、boxes 和 scores。

Listing 5 单帧图像文本提示调用

```
1 model = build_sam3_image_model(
2     checkpoint_path=str(args.checkpoint),
3     load_from_HF=False,
4     compile=False,
5 )
6 processor = Sam3Processor(
7     model,
8     confidence_threshold=args.confidence_threshold,
9 )
10
11 state = processor.set_image(image)
12 output = processor.set_text_prompt(state=state, prompt=case.prompt)
```

这个脚本用于观察 SAM 3.1 在非视频场景下的开放词汇分割表现。报告中保留 building、truck、wheel 等成功样例，同时记录 tree 和 bottle 在默认阈值下没有保留实例的情况，用作提示词和阈值敏感性的对照。

第四章 实验环境与运行流程

4.1 环境配置命令

以下是环境搭建用到的关键命令。安装前确认当前 shell 位于 base 环境，但所有依赖均通过 conda run -n sam31 安装到独立环境。

Listing 6 环境配置命令摘录

```
1 conda create -n sam31 python=3.12 -y
2
```

```

3 conda run -n sam31 python -m pip install \
4     torch==2.6.0 torchvision==0.21.0 \
5     --index-url https://download.pytorch.org/whl/cu124
6
7 conda run -n sam31 python -m pip install -e . \
8     modelscope matplotlib opencv-python-headless imageio
9
10 conda run -n sam31 python -m pip install \
11     einops pycocotools psutil pandas scikit-image scikit-learn

```

由于 SAM3 要求 `numpy<2`，且源码仍使用 `pkg_resources`，实验中固定了两个兼容依赖：

Listing 7 兼容依赖固定

```

1 conda run -n sam31 python -m pip install \
2     --force-reinstall --no-deps numpy==1.26.4
3 conda run -n sam31 python -m pip install "setuptools<81"

```

4.2 权重下载

SAM 3.1 checkpoint 从 ModelScope 下载，避免 Hugging Face gated checkpoint 登录问题。下载命令如下：

Listing 8 ModelScope 权重下载命令

```

1 mkdir -p checkpoints/sam3.1
2 conda run -n sam31 modelscope download \
3     --model facebook/sam3.1 \
4     sam3.1_multiplex.pt config.json \
5     --local_dir checkpoints/sam3.1

```

下载完成后，`sam3.1_multiplex.pt` 文件大小为 3,502,755,717 字节，SHA256 为 0567debeec80ba4ac6369540c6c248025283cb3ff2b92827509e57e2b3541cb6，与 ModelScope LFS 指针一致。

4.3 实验运行命令

三组实验均使用官方 `assets/player.gif` 抽取出的 8 帧序列，帧步长为 6。

Listing 9 三组视频 prompt 实验命令

```

1 PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
2 conda run -n sam31 python scripts/run_sam31_player.py \
3     --max-frames 8 --frame-step 6 \
4     --prompt person --output-prefix sam31_player_person
5

```

```

6 PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
7 conda run -n sam31 python scripts/run_sam31_player.py \
8     --max-frames 8 --frame-step 6 \
9     --prompt ball --output-prefix sam31_player_ball
10
11 PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
12 conda run -n sam31 python scripts/run_sam31_player.py \
13     --max-frames 8 --frame-step 6 \
14     --prompt "player in red" --output-prefix sam31_player_red

```

单帧图像补充实验使用同一个 checkpoint，但走图像 predictor，不创建视频 session：

Listing 10 单帧图像补充实验命令

```

1 conda run -n sam31 python scripts/run_sam31_image_cases.py \
2     --confidence-threshold 0.35 --max-instances 8

```

脚本生成的文件按结果类型分开保存。表 4 列出报告中使用的主要输出，后续复现实验时可以直接对照这些文件检查。

表 4 实验输出文件说明

文件	来源脚本	内容
sam31_player_*_metadata.json	run_sam31_player.py	三组视频实验的帧数、耗时、GPU 和逐帧目标数。
sam31-video-prompt-grid.png	报告整理图	三组视频 prompt 在第 0、4、7 帧的九宫格对比。
sam31_image_cases_metadata.json	run_sam31_image_cases.py	单帧图像实验中各 prompt 的目标数、耗时和最高置信度。
sam31-image-cases-grid.png	报告整理图	单帧图像成功样例和失败样例说明。

第五章 多组实验结果与对比分析

5.1 视频实验结果汇总

三组视频结果汇总在表 5 中。每组都顺利完成 8 帧传播，没有出现空白输出。图 4 将三组 prompt 在第 0、4、7 帧上的输出统一排版，便于直接比较类别提示、小目标提示和属性提示之间的差异。

表 5 三组文本提示词实验结果

Prompt	每帧目标数	加载时间	推理时间	观察结果
person	3	9.35 s	1.46 s	检测并跟踪三名球员，目标 id 在不同帧中保持稳定。
ball	1	9.47 s	1.43 s	检测并跟踪足球，小目标位置能够随帧变化更新。
player in red	1	9.42 s	1.49 s	只选择红色球员，体现文本属性约束能力。



图 4 三组视频文本提示词的跨帧分割结果对比。每一行对应一个 prompt，每一列对应同一视频中的一个采样帧。

Listing 11 player in red 实验 metadata 摘录

```

1 {
2   "prompt": "player in red",
3   "num_frames": 8,
4   "selected_frames": [0, 4, 7],
5   "load_seconds": 9.42,
6   "inference_seconds": 1.49,
7   "objects_per_frame": {
8     "0": 1,
9     "1": 1,
10    "2": 1,
11    "3": 1,
12    "4": 1,
13    "5": 1,
14    "6": 1,

```

```
15     "7": 1
16   }
17 }
```

metadata 中记录了 prompt、采样帧、模型加载耗时、推理耗时和每帧目标数。以上耗时数据均由脚本自动记录至 metadata JSON 文件，随后整理为表格。

5.2 粗粒度类别提示：person

使用 person 提示时只给出人物类别，没有限制颜色或位置。模型将画面中符合人物概念的多个实例都找了出来，因此每帧得到 3 个目标。图 4 第一行可以看到，白衣球员、红衣球员和远处球员被分配不同颜色和 id，目标集合在后续帧中保持稳定。

5.3 小目标提示：ball

足球面积小，和草地背景颜色也比较接近，属于这段视频中较难稳定跟踪的目标。使用 ball 提示时，模型仍能在每帧中保持 1 个足球目标输出。图 4 第二行显示足球框会随运动位置更新，但 mask 面积较小，说明小目标结果对分辨率和置信度阈值更敏感。

5.4 细粒度属性提示：player in red

player in red 在类别基础上加入了颜色属性约束。与 person 不同，该提示并不要求输出全部人物，而是只锁定红色球衣球员。图 4 第三行显示，模型每帧输出 1 个目标，并优先锁定红衣球员。这说明 SAM 3.1 的文本提示不仅能表达类别，也能表达颜色属性等细粒度概念。

5.5 单帧图像补充实验

为进一步验证模型在不同场景下的泛化能力，本实验补充选取校园夜景、车厢内日常物品场景和街景货车图片进行单帧开放词汇分割。图像实验共加载一次模型，加载时间为 6.12 s；每个 case 独立调用 set_image 和 set_text_prompt。结果见表 6 和图 5。

表 6 单帧图像文本提示实验结果

图像场景	Prompt	目标数	推理时间	最高分	观察结果
校园夜景	building	11	0.28 s	0.859	能找出主楼和远处建筑，但夜景噪声使部分检测框较为零散。
校园夜景	tree	0	0.09 s	—	树木与暗背景区分度低，默认阈值下未保留实例。
车厢物品	bottle	0	0.09 s	—	目标被纸袋遮挡且外观不明显，未形成稳定候选。
街景货车	truck	1	0.09 s	0.875	能完整覆盖整车主体。
街景货车	wheel	4	0.09 s	0.949	能响应部件级提示，但也会包含车轮附近相似部件。



图 5 单帧图像文本提示补充实验。成功样例覆盖场景级建筑、整车目标和车轮部件；右下角记录默认阈值下未保留实例的对照样例。

5.6 结果对比小结

视频实验和单帧实验共同说明了 SAM 3.1 的开放词汇能力。类别提示更容易召回多个实例，例如 person 和 building；细粒度提示能将结果聚焦到特定目标，例如 player in red 和 wheel；小目标或遮挡目标更依赖图像清晰度和阈值设置，例如 ball、tree 和 bottle。因此，SAM 3.1 适合快速完成自然语言驱动的分

第六章 问题处理与总结

6.1 运行中遇到的问题

实验过程中主要处理了环境版本、依赖缺失、接口兼容和阈值选择四类问题。官方 README 推荐 CUDA 12.8 PyTorch，但本机驱动显示 CUDA 12.4，因此实际采用 `torch==2.6.0+cu124`。SAM3 源码导入 `pkg_resources`，新版 `setuptools` 会导致模块缺失，所以固定 `setuptools<81`。此外，官方基础依赖没有完全覆盖 video predictor 的实际运行路径，需要额外安装 `einops`、`pycocotools`、`psutil` 等包。SAM3.1 multiplex 模型的 `init_state` 参数签名与基类 `start_session` 传入的参数不匹配，本实验脚本通过兼容启动函数规避了该问题，未改动官方源码。单帧图像实验中，`tree` 和 `bottle` 在默认阈值下没有输出，也说明开放词汇分割结果仍会受图像质量、遮挡和 `confidence threshold` 影响。

6.2 实验结论

本实验在本地 RTX 4090 上跑通了 SAM 3.1 的开放词汇视频分割、多目标跟踪和单帧图像提示分割。三组视频提示词分别验证了不同能力：`person` 对应多实例类别分割，`ball` 对应小目标定位，`player in red` 对应属性约束下的细粒度检索；单帧图像实验进一步展示了 `building`、`truck`、`wheel` 等不同粒度 prompt 的输出差异。从代码角度来看，SAM 3.1 把 `detector`、`tracker` 和 `multiplex memory` 等复杂模块封装在 `session` 请求接口后面，对外调用足够简洁，这也是本实验选用它作为图像处理与计算机视觉课程项目的原因。

6.3 后续改进

后续可进一步增加自采视频或公开数据集样例，并比较 SAM 3、SAM 3.1 和 SAM 2.1 在相同视频上的速度与精度差异。单帧实验可以增加不同 `confidence threshold` 的扫描，观察召回率和误检之间的变化。如果环境支持 CUDA 12.8 和 FlashAttention 3，也可以重新开启 `use_fa3` 与 `torch.compile`，评估 Object Multiplex 的实际加速效果。

参考文献

- [Meta AI Research(2026a)] Meta AI Research, . SAM 3: Segment Anything with Concepts. <https://github.com/facebookresearch/sam3>, 2026a. Accessed: 2026-06-07.
- [Meta AI Research(2026b)] Meta AI Research, . Segment Anything Model 3 Project Page. <https://ai.meta.com/sam3>, 2026b. Accessed: 2026-06-07.
- [Bu(2026)] Bu, W. SAM 3.1 Vision Homework Experiment Materials. <https://github.com/muqy1818/sam31-vision-homework>, 2026. Accessed: 2026-06-08.
- [Meta AI Research(2026)] Meta AI Research, . SAM 3.1 Release Notes. https://github.com/facebookresearch/sam3/blob/main/RELEASE_SAM3p1.md, 2026. Accessed: 2026-06-07.
- [PyTorch Contributors(2026)] PyTorch Contributors, . PyTorch Installation Documentation. <https://pytorch.org/get-started/locally/>, 2026. Accessed: 2026-06-07.
- [ModelScope(2026)] ModelScope, . facebook/sam3.1 Model Repository. <https://www.modelscope.cn/models/facebook/sam3.1>, 2026. Accessed: 2026-06-07.